Exploring Accuracy with Serendipitous Discoveries in Recommender Systems

Nazneen Fatema Rajani
The University of Texas at Austin
Austin,TX USA
nrajani@cs.utexas.edu

ABSTRACT

This paper introduces a system for movie recommendations implemented using information retrieval and cluster analysis based on a hybrid recommendation approach. Most content-based recommenders rely on the similarity between the bought, searched or visited items and all the other items in the repository. Collaborative filtering algorithm is based on the assumption that a user is interested in what is similar to what he/she has bought, searched or visited [8]. There are some contexts in which this assumption can be violated. Serendipity is the case of acquiring unsearched but still useful items or pieces of information.

This paper presents the design and implementation of a hybrid recommendation approach that joins the content-based and serendipitous discoveries. The system makes reasonable, yet surprising, horizon-broadening movie recommendations. This paper considers MovieLens corpus of 100,000 ratings (1-5) from 943 users on 1682 movies.

Keywords

Content-based Collaborative Filtering, Serendipitous discoveries, Hybrid recommender systems

1. INTRODUCTION

Consumers today are faced with information overload and the problem of too much choice, for example there are close to 75K movies on Netflix, which means its very difficult for a user to make a good choice on the movies. In order to cope up with this information explosion, several companies today have deployed recommendation systems to help their consumers make good choices. Recommender Systems are vital because they provide a win-win situation wherein users find items of interest to them and e-commerce enterprises increase sales and user satisfaction. Recommender Systems have also become an important research area since the mid-1990s [2]. Much work has been done on developing new approaches to recommender systems both in academia and in industry. However, despite these advances, the current generation of recommender systems still requires further improvements in order to accomodate demands in broader areas like traveling, entertainment, shopping.etc. These improvements include more effective recommendations spread across diverse items and better methods for representing user behavior. There is also demand for improvement in the information about the items to be recommended, more advanced recommendation modeling methods, incorporation of various contextual information into the recommendation process, utilization of multi-criteria ratings, development of less intrusive and more flexible recommendation methods that also rely on the measures that more effectively determine performance of recommender systems.

This paper briefly describes and explores the capabilities of three main categories of recommender systems: content-based, collaborative and hybrid in section 3. Section 4 discusses a hybrid movie recommendation algorithm which joins the traditional content-based collaborative filtering technique with 'surprise-me' recommendations not in the proximity of a user's preferences and thus enabling them with serendipitous discoveries [15]. This paper shows that hybrid recommendation systems perform better than the traditional content-based approach [14]. Since the hybrid approach makes recommendations beyond a user's preferences, it serves in broadening the horizon of users. In particular, the content-based approach suffers from over-specialization because it only recommends movies that score highly against a user's profile and the user is limited to being recommended movies similar to those already rated. The data-set considered is MovieLens corpus of 100,000 ratings (1-5) from 943 users on 1682 movies in which each user has rated at least 20 movies [1]. The data-set also gives information on the genre a movie belongs to. A movie may belong to one or more of the following 19 genres – Action, Adventure, Animation, Children's, Comedy, Crime, Documentary, Drama, Fantasy, Film-Noir, Horror, Musical, Mystery, Romance, Sci-Fi, Thriller, War, Western or unknown.

2. RELATED WORK

This section briefly reviews related work which can be categorized into graph mining and recommender systems. There is little work in recommender systems which focuses on broadening user's interest. The work of Kamahara et al. [9], who explore methods for locating unexpected items from similar clusters to user's cluster, is close to the posed problem. However, parameters such as the number of clusters and thresholds, which are required in [9], are not easy to be tuned. Bell et al. [3] propose a low rank matrix approximation method for collaborative filtering and successfully apply it to the NetFlix competition. Fouss et al. propose in [5] to apply Euclidean distance, which is one of random walk based methods of computing similarities between nodes, to a collaborative recommendation. Recommender systems have been attracting considerable research interest, with recent emphasis on graph-based systems. Celma et al [4] presented two complementary methods to an-

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
Copyright 200X ACM X-XXXXX-XX-X/XX/XX ...$10.00.

alyze and evaluate novel recommendations. The item-centric evaluation method involves analyzing the item-based recommendation network to detect whether the intrinsic topology of the network has a pathology that hinders novel recommendations. The user-centric evaluation aims to measure the perceived quality of the recommendations. Although there are differences in the way specific methods are implemented, the basic idea behind all these methods is to find items that are most relevant to the older choices of the given user.

3. TYPES OF RECOMMENDER SYSTEMS

The recommendation problem can be formulated as follows: Let S be the set of all users and let T be the set of all possible items that can be recommended, such as books, movies, or restaurants. This space T of items can be very large, ranging in millions in certain cases. Similarly, the user space S can also be large. The utility function u can be defined as the the usefulness of item t to a user s. In recommender systems, this utility is generally the rating that a user would have given for a particular item. Then, for each user $s \in S$, we want to choose such items $t \in T$ that maximize the user's utility. This is a challenging task for two reasons:

- Predicting ratings for individual items is imperfect and at best yields noisy estimates. Therefore, if we wish to estimate opinion of a specific individual about a specific item, then we need to find some other information that helps us average out the noise prediction of ratings.
- If an individual has not rated an item, then there is no direct evidence of the sentiment. In other words, we have to figure out a way to extrapolate the opinion of that individual from items he/she has rated to unseen or not-yet-rated items.

Recommender systems are usually classified according to their approach to rating estimation into the following categories:

- **Content-based recommendations**: Recommends items that are similar to what the user has preferred in the past.
- **Collaborative recommendations**: Recommends items that people with similar tastes have preferred in the past.
- **Hybrid approach**: Combines content-based and collaborative approaches along with heuristics that estimate the utility function by optimizing certain performance criterion like the mean square error.

3.1 Content-Based Recommendations

In content-based recommendations, the utility $u(s,t)$ of item t for user s is estimated based on the utilities $u(s,t_i)$ assigned by user s to items $t_i \in T$ that are "similar" to item t. For example, in a movie recommendation system, in order to recommend movies to user s, the content-based recommender application tries to understand the similarities (like specific actors, directors, genres, storyline, etc.) among the movies user s has rated highly in the past. Then, only the movies that have a high degree of similarity to whatever the user's past preferences are would be recommended.

3.1.1 Drawbacks

The content- based approach is a traditional method for making recommendations and has several drawbacks [2], some of which are discussed below:

- **Limited Content Analysis**: Content-based approach is limited by the features that are explicitly associated with the items that these systems recommend. If the item to be recommended is a movie, this approach may not face difficulties because movies in general have several features (like actors, directors, genres, storyline,etc.) that can be parsed automatically. However if the item to be recommended is multimedia data like graphical images or audio stream, it is not only difficult to define specific features for these items abut also hard to parse them automatically. Limited content analysis cannot correctly classify two different items represented by the same set of features because it cannot distinguish between them [13]. This problem mainly arises in text-based documents because they are usually represented by their most important keywords and content-based systems cannot distinguish between a well written and badly written document having same keywords.
- **New User Problem**: It is impossible for content-based applications to make accurate recommendations to a new user or a fairly new user who does not have sufficient history of ratings/preferences.
- **Overspecialization**: Content-based recommendations restrict a user's choice to items that are similar to those that are already rated. Thus a user who has never rated a comedy movie will never be recommended even the best comedy movie. Another problem with overspecialization is that recommending items that are too similar would lead to the user having the exact same experience which may not always be desirable. For example, while recommending news articles, it would not be desirable to be recommended articles that provide the same information.

3.2 Collaborative Recommendations

Collaborative filtering recommender systems predict the utility of items for a particular user based on the items previously rated by other users. Thus the utility $u_{s,t}$ of item t for user s is estimated based on the utilities $u_{s,t}$ assigned to item t by the users $c_j \in C$ who are "similar" to user c. For example, in a collaborative movie recommendation application, the user is recommended movies that are similar to the movies rated by other users or "peers" whose tastes overlap with the current user's. Note that the similar tastes may not be restricted to movies, overlapping likes/dislikes on any items may be considered for recommending movies using collaborative approach. A challenging task while using this approach is to identify "peers" who share tastes with the user and these are generally people in the social network of the user. The most widely used method to identify peers is to use the nearest neighbor search in the social network of a user. This approach calculates a similarity score $sim(s,s')$ between the user and other users; higher the score, higher degree of similarity between the users. The most common approach to calculate $sim(s,s')$ is to find the cosine of the angle between the vectors formed by the ratings of the two users on common items. This has been discussed in depth in section 4.

3.2.1 Drawbacks

Since the basic idea behind collaborative filtering system is similar to the content-based approach, they also share certain drawbacks, however collaborative approach can deal with any kind of content and recommend any items, even the ones that are dissimilar to those seen in the past.

- **New User Problem**: Collaborative filtering technique for recommendation relies on the history of the user's preferences and thus faces the exact same problem as the content-based approach for a new or fairly new user with little or no past preferences. Fortunately hybrid recommender systems

do not face this problem and is one of the reasons why the paper considers the hybrid approach to recommending movies.

- **New Item Problem**: New items keep getting added to recommender systems each day. Since collaborative systems rely solely on user's preferences to make recommendations, until the new item is rated by a substantial number of users, the recommender system would not be able to recommend it. Again, the hybrid recommender system does not suffer from this drawback.

- **Sparsity**: In any recommender system, the matrix of users and items is a sparse one, this is because the number of ratings already obtained is usually very small compared to the number of ratings that need to be predicted. In order to perform well, a recommender system must be able to effectively extrapolate the sparse matrix to a dense one. Also, for the user whose tastes are uncanny compared to the rest of the population, there will not be any other users who are particularly similar, leading to poor recommendations for that user.

3.3 Hybrid Approach

To overcome the drawbacks on content-based and collaborative filtering approaches to recommender systems, hybrid recommender system is presented in this section. As the name suggests, these combine the traditional approaches to recommendations with heuristics that estimate the utility function. There are different ways in which these approaches can be combined into a hybrid recommender system which can be classified as follows:

1. Implementing collaborative and content-based methods separately and combining their predictions,

2. Combining either collaborative or content-based methods with a heuristic method for improving predictions, and

3. Constructing a unifying model that incorporates both content-based and collaborative characteristics along with a heuristic approach to estimating utility that optimizes performance.

The first one generally uses a linear combination of the outcome ratings of both methods to predict a final rating. The second method incorporates a heuristic evaluation with the traditional approaches to improve the quality of the results. The heuristic generally provides more information about either the item or the user or both. For example, if a user's profile (age, gender, demographics) are know aprior, this information can be used to improve the quality of recommendations made. The third method combines all three approaches and thus performs significantly better; needless to say this paper follows the third approach of hybrid recommender systems. The heuristic used in this paper for improving accuracy estimates the utility of serendipitous discoveries in recommender systems. This has been discussed in detail in section 4.

4. RECOMMENDATION ALGORITHM

This section discusses the hybrid approach used for recommendation on MovieLens Corpus. Initially, the basic framework of the system is based on the traditional content-based collaborative filtering approach which unifies the first two categories of recommender systems is discussed in section 3 and is the baseline method. Thereafter, we present an approach that enables serendipitous discoveries in recommender systems and prove that this approach estimates the utility function that increases the accuracy of the system. Finally both the methods are merged to obtain the required hybrid movie recommendation system [6].

4.1 Content-Based Collaborative Filtering

The content-based collaborative filtering technique is used as a basic framework because of the sparsity of MovieLens ratings corpus. In this approach, the movies are represented as vectors in user space, where the coordinates represent the ratings given by a user on a movie. The aim is to predict the ratings for movies in the corpus for a query user and recommend the ones with the higher ratings. Rating $r(i, j)$ corresponds to a rating by a query user i for movie j. The mean rating for user i is:

$$\bar{r_i} = \frac{1}{|M_i|} \sum_{j=1}^{M_i} r_{i,j} \tag{1}$$

where M_i is the set of movies the user i has rated on.

In a content-based approach, a prediction for a active user is derived by considering all his/her past ratings while a collaborative-based approach uses past ratings of other users. Each query user is compared to every other user for similarity of preferences. To achieve this end, the approach is to identify k such similar users called the "neighbors" of the query user. The predicted rating using content-based collaborative filtering for the active user a on movie j, $R_{a,j}$, is a weighted sum of ratings of other similar users.

kNN is the set of k nearest neighbors of the current user. A neighbor has a high similarity value $sim(a, i)$ with the current active user. There are various ways to calculate this similarity score for finding the k nearest neighbors. The method used in this paper is called locality sensitive hashing (LSH). This method hashes each query user into a bucket such that the probability of collision is higher for users that are nearby. When a new user arrives, it is hashed into a bucket and the users in the same bucket are inspected and nearest one returned.

Table 1: User Ratings Matrix

Users	Movie Ratings						
User 1	5	0	0	0	2	0	0
User 2	0	1	0	0	0	4	5
User 3	1	0	4	0	0	3	0
User 4	0	0	0	0	1	0	0
User 5	2	0	1	1	0	0	0

The crucial part of the algorithm is the ability to determine a set of users who are most relevant or similar to the active user for whom the recommendation is being sought, $sim(a, i)$ [10]. Thus, it is required to compare the active user with each user in the repository in order to determine the kNN nearest users. A matrix of user ratings is created as shown in Table 1. In LSH the probability of two points A and B colliding is proportional to cosine of the angle between them [12], equation 2.

$$\cos\theta = \frac{A \cdot B}{\|A\|\|B\|} \tag{2}$$

More precisely, the probability that two points collide under the LSH hashing scheme is given by equation 3.

$$P_{col} = 1 - \frac{\theta(A, B)}{\pi} \tag{3}$$

While making a recommendation for the active user, a query vector of the current user is created and compared with all other user vectors. In order to make sure that non-similar users are not hashed

in the same bucket, it is necessary to decrease the probability of collision of two non-similar points. This can be ensured by choosing an optimal k, which is the number of hyperplanes in the user space. Higher the k, fewer collisions in buckets but more time to compute the hash values. However, fewer collisions also means fewer collisions with nearest neighbor which is not desirable. An optimal value of k = 8 has been chosen as a reasonable balance between time spent computing the distances and the time spent computing the hash functions.

Locality Sensitive Hashing is widely used in several applications involving nearest neighbors, however, Algorithm 1 is an innovative approach to calculate the similarity score using the LSH method for finding the nearest neighbor given a query user vector. The parameters used are k and number of hash tables L. Given k and small value δ which is the desired probability of missing a nearest neighbor, the number of hash tables is computed in equation 4.

$$L = log_{(1-P_{col})}\delta \tag{4}$$

Algorithm 1 Locality Sensitive Hashing (LHS) Algorithm

```
for user u in the corpus do
    add u to LSH
    S ←set of points that collide with u in LSH
    sim_min (u) ← 1
    for user u' in S do
        t = distance (u, u')
        if t < sim_min (u) then
            sim_min (u) ← t
        end if
    end for
    assign score sim_min (u) to u
end for
```

The set of neighbors is sorted in descending order of calculated similarity score. The predicted ratings for a query user are then calculated as in equation 5 wherein N is the normalizing factor such that the absolute values of the scores sum to unity.

$$R_{a,j} = \bar{r_a} + N \sum_{i=1}^{kNN} sim(a,i)(r_{i,j} - \bar{r_i}) \tag{5}$$

Movies are ranked based on their predicted rating and the top $n = 5$ movies are recommended to the user. Although content-based collaborative filtering algorithms have potential implications for scalability and runtime performance, in general however, the algorithm works reasonably well, and is straightforward to implement.

4.2 Serendipitous Discoveries in Recommendations

This section presents a heuristic that enables serendipitous discoveries in recommender systems. At this point, it would be useful to differentiate between novel and serendipitous recommendations. A novel recommendation is the one which helps users find interesting items he/she might have autonomously discovered while a serendipitous recommendation is the one which helps users find interesting items he/she might not have otherwise discovered. As discussed before using such an algorithm precludes the need for collecting large user data of past ratings. The algorithm works effectively even for users who do not have history of any preferences or interests, this is because the algorithm assumes that some

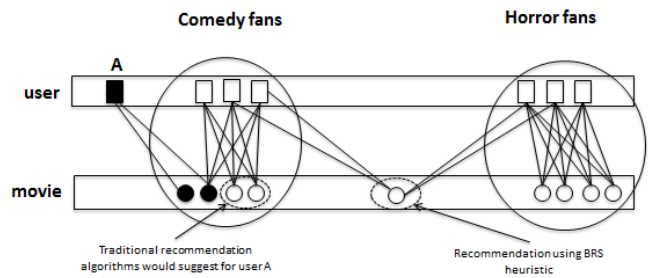


Figure 1: Conventional recommendation algorithms versus BRS heuristics

items have high levels of coherence to variety of interests; in case of MovieLens data, these interests are the movie genres. For example a movie like Star Wars (1977) which belongs to several genres like action, adventure, sci-fi, romance and war has a tendency to attract a larger audience than a movie which belongs to a very specific genre.

In order to enable serendipitous discoveries, this paper estimates the utility function for a "surprise me" query: A user may be bored with his/her usual genre of movies, and may want a recommendation that is related, but off the beaten path, possibly leading to a new genre of movies. The main idea behind this approach is to envision the problem as node selection on a graph, giving high scores to nodes that are well connected to the older choices, and at the same time well connected to unrelated choices. Consider a bipartite graph of users and movies wherein each node in the graph represents either a movie or a user and the users are connected to movies they like with an edge that has weight proportional to the rating given by that user for that movie, however the link is unidirectional so the weights are symmetric. The heuristic used in the proposed algorithm is called the Bridging Score (BRS). Bridging score means the degree of connectivity to other groups on a graph. One approach to calculate BRS would be to consider that popular items have a potential to bridge groups, but popularity does not always correspond to the connectivity to other groups.

The paper now proposes an algorithm to calculate the bridging score that takes into account the connectivity to other groups in the graph in order to broaden a user's horizons. It is based on the graph mining technique of computing similarity between nodes and can be applied to any dataset that can be represented as a graph; Figure 1 illustrates the difference in recommendations of traditional approaches versus the BRS heuristic.

Let l_i be the number of edges connected to node i. Given a node i find the set of nodes to which i links: $S_i = \{s_{i,j} | 0 \leq j \leq l_i\}$. Then compute the similarity scores (using Algorithm 1) between any pair of elements in S_i and construct a $l_i \times l_i$ similarity matrix R_i . After that, take the mean of all non-diagonal elements in R_i and get b_i by inverting it. If node i is connected by only one edge or it has no connection, define its bridging score as b_i=0, which means that it is not taken into account as a candidate of recommendation. If i is connected to one group, then the similarity scores between any pair of elements in S_i should be high and as a consequence, b_i becomes small. On the other hand, if i has links to several groups, the similarity scores between different groups have small numbers and we get large b_i. Onuma et al. in [11] have put forth Algorithm 2 that summarizes bridging score computation. Once the vector of bridging scores is computed, sort the vector in descending order of b_i; and recommend movies linked with higher bridging scores.

Algorithm 2 Bridging Score (BRS) Computation

Given: User-Noun Matrix X
for $i = 1 \rightarrow n$ **do**
  Let $S_i = \{s_{i,j} | 0 \leq j \leq l_i\}$ be set of nodes which have links to i;
  if $(l_i = 0 \text{OR} l_i = 1)$ **then**
    $b_i = 0$;
  else
    Compute the similarity score vectors $r_{s,1}, r_{s,2}, ..., r_{s,l_i}$
    for each $S_i = s_1, s_2, ...s_{l_i}$;
    Construct $l_i \times l_i$ matrix $R_i = [r_{s,1}, r_{s,2}, ..., r_{s,l_i}]$;
    Take the average of all non-diagonal elements in R_i
    to obtain $\hat{r_s}$;
    $b_i = 1/\hat{r_s}$;
  end if
end for
return vector b of bridging scores;

4.3 Hybrid Recommender System: Merging content-based CF and Serendipitous Discoveries

This section deals with merging the BRS heuristic with content-based collaborative filtering algorithm for recommendations. In order to build a hybrid recommender system, the goal is to join the content-based CF which generates a similarity score with BRS heuristic which generates a bridging score to enable relevant but serendipitous discoveries. There are several ways to combine the above two criteria. However reasonable-looking choices may suffer from subtle issues which have been discussed here. Let the vector of similarity score to a query node Q be denoted by sim_Q and the bridging score vector be denoted by b_Q, t_Q be the hybrid classifier score.

The first natural method is to try a linear combination of similarity score vector and the bridging score vector as in equation 6.

$$t_Q = \alpha_1 \times sim_Q + \alpha_2 \times b_Q \tag{6}$$

where α_1 and α_2 are the coefficients and atleast one of them is non-zero. However the above combination suffers from two problems: (a) the similarity score and the bridging score typically differ by orders of magnitude, and the larger, b_j , typically overshadows sim_Q, (b) a good value of α_1 and α_2 need to be determined which may need to be changed as the graph grows.

The second most reasonable method of combination is multiplication: i.e. $t_Q = sim_Q \times b_Q = sim_Q/\hat{r_s}$. It can be observed that t_Q represents the ratio of similarity score to query nodes and similarity score among neighbors. Also this combination is parameter free and hence is a good way to merge content-based CF with BRS heuristic.

5. EVALUATION METRIC

The approach discussed in making movie recommendations uses a hybrid of conventional recommendation algorithms and bridging score heuristic for serendipitous discoveries. This section discusses the evaluation metric used in computing the efficiency of the proposed algorithms. To emphasize the the accuracy of hybrid recommender systems over recommender systems that use traditional approaches, it is first required to evaluate the performance of content-based collaborative filtering algorithm on MovieLens data-set and contrast it with the performance of a hybrid recommender system that joins the conventional approach with BRS heuristic.

5.1 Content-Based Collaborative Filtering

The metric used for evaluating the performance of baseline method which is the content-based collaborative filtering approach to recommendations on MovieLens data-set is the Root of the Mean Square Error (RMSE). As shown in Table 1, the dataset is a sparse user-movie matrix. To explore whether the recommendation algorithm can predict entries which are ratings in this matrix, hide K non-zero entries (i.e. set the entries to zero) and make predictions for these entries. If $p_{i,j}$ is the predicted rating for user i over item j, and $v_{i,j}$ is the true rating, and $K = (i, j)$ is the set of hidden user-item ratings then the RMSE is defined as:

$$\text{RMSE} = \sqrt{\frac{\sum_{(i,j)\in K} (p_{\text{i,j}} - v_{\text{i,j}})^2}{n}} \tag{7}$$

RMSE is suitable for the prediction task, because it measures inaccuracies on all ratings, either negative or positive. However, it is most suitable for situations where it is not required to differentiate between errors. For example, in the movie rating prediction, it may not be as important to properly predict the difference between 1 and 2 stars as between 2 and 3 stars. If the system predicts 2 instead of the true 1 rating, it is unlikely that the user will perceive this as a recommendation. However, a predicted rating of 3 may seem like an encouragement to watch the movie, while a prediction of 2 is typically considered negative. Herlocker et al. [7] show that the space of ratings is not truly uniform, and that it can be mapped to a uniform space to avoid such phenomena.

Another metric used for evaluation is the classification error (CE) which is the number of mis-classifications. This is obtained as follows: The original user-noun ratings matrix and the predicted user-noun ratings matrix are converted to binary ratings using certain rating value as a threshold. Thus the problem is converted into a binary classification problem and CE is the number of mis-classifications between the true hidden entries and the predicted entries.

5.2 Hybrid Recommender System

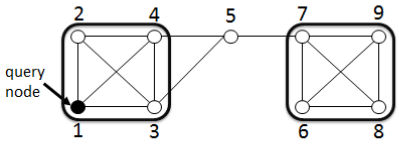
An important observation to note is that it is not only difficult but also impossible to measure the accuracy of a "surprise-me" recommendation because it is highly subjective and may vary from individual to individual. However, it is possible to calculate the accuracy of recommendations that are relevant as well as contain an element of surprise which is precisely what the proposed hybrid classifier score t_Q calculates. Let n_{hyb} and n_{conv} be the surprising nodes in the returned recommendation list by the hybrid classifier and by conventional recommendation algorithms respectively. A useful metric is the "Surprising Gain" which is defined as $\frac{n_{hyb}-n_{conv}}{n_{conv}}$. A big " Surprising Gain" means that the hybrid classifier returns more surprising results compared with conventional recommendation algorithms. Such an evaluation of the hybrid classifier on synthetic dataset has been discussed in section 6.2.

6. RESULTS AND ANALYSIS

This section deals with the performance and analysis of the conventional content-based CF algorithm vs. the proposed hybrid recommender approach. To emphasize on the effectiveness of such a hybrid system, the section also discusses the working of the proposed algorithm on some synthetic datasets.

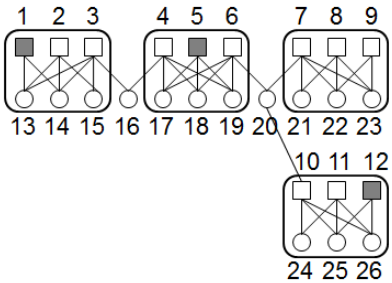
6.1 Content-Based Collaborative Filtering

The baseline method used in the hybrid algorithm was the content-based collaborative filtering algorithm implemented on MovieLens



node	$sim_{1,i}$	$b_{1,i}$	$t_{1,i}$
1	0.621	8.579	5.327
2	0.219	8.579	1.878
3	0.201	11.085	2.228
4	0.201	11.085	2.228
5	0.187	20.089	3.756
6	0.101	7.967	0.804
7	0.150	12.847	1.927
8	0.101	7.967	0.804
9	0.101	7.967	0.804

Figure 2: Hybrid classifier recommendations on a synthetic dataset 1



Query Node	Hybrid Classifier Recommendation
node 1	node 16
node 5	node 20
node 12	node 20

Figure 3: Hybrid classifier recommendations on a synthetic dataset 2

corpus and results were analyzed. The recommendations as discussed earlier were restricted to $n = 5$, thus when given a user a the algorithm made recommendations for top 5 movies based on the predicted movie ranking for that user.

In the previous section, the user-ratings matrix was generated, let it be called X. Next step is to explore whether the algorithm can predict entries in the matrix X. As discussed, about 30% non-zero entries are randomly hidden from X and predictions are made on the value of these hidden entries based on the remaining 70% entries. The aim is to predict the sign of the hidden entries which is related to the similarity score [11]. The process of hiding random matrix entries, predicting them and calculating the RMSE values using equation 7 is repeated twenty times. The mean RMSE obtained is 0.87. By converting the ratings to binary using a threshold of 2.5, the problem is transformed into binary classification problem. The process of hiding random matrix entries, predicting the entry and calculating the classification error (CE) is repeated for twenty iterations. The mean CE obtained using content-based CF is 19%. That means the algorithm is accurate to 81% which is far better than making a random recommendation. Therefore, it is assured that the baseline method performs fairly accurately in making recommendations to a query user.

Consider a user who is a fan of comedy movies, Table 2 shows a list of movies that the user has given a rating of 5 stars. Table 3 illustrates the top five recommendations made by the content-based CF algorithm for this user.

6.2 Hybrid-Recommendation System's Evaluation on Synthetic Dataset

To visually imagine how the hybrid classifier works and produces reasonable recommendations, it would be best to use it on a synthetic dataset much smaller in size. The main idea behind doing this is to empower the reader to imagine how the hybrid classifier works geometrically to produce reasonably accurate yet surprising recommendations. Consider the graph in Figure 2, which has two groups; nodes 1 - 4 in Group 1 and nodes 6 - 9 in Group 2, which are connected through node 5, now run a query for node 1. Node 2 gets the highest similarity score among nodes except the query node, followed by nodes 3 and 4. This result would be same as what conventional recommendation algorithm would give. On the other hand, comparing bridging scores, it is found that node 5 gets the highest score, because the neighbors of node 5 (nodes 3, 4, 6 and 7) are separated in half into two groups and, as a consequence, the average of similarity scores among them becomes small. Notice that nodes 7, 3 and 4 also get relatively high bridging scores. As a result, nodes 3 and 4 are ranked as No. 1 by hybrid classifier score t_Q, as reflected by intuition that these nodes are close enough to the query node as well as closely connected to Group 2.

Consider example of another graph in Figure 3, where square nodes represent users, circular nodes denote movies, and edges correspond to ratings by users. Some fascinating observations made are: (a) movie 16 gets the highest score for user 1 by the hybrid classifier. This result completely agrees with the motivation described in Figure 2. (b) The hybrid classifier concludes movie 20 to be the best recommendation for user 5, not movie 16. It is a reasonable result because movie 20 is watched by three groups; while movie 16 is watched by two groups.

Table 2: Movies rated 5 stars by a comedy fan

Movies	Genre
Fatal Instinct(1993)	Comedy
Young Frankanstein(1974)	Comedy, Horror
Naked Gun 33 1/3: The Final Insult(1994)	Comedy
Robin Hood: Men in Tights	Comedy

6.3 Hybrid-Recommendation System's Evaluation on MovieLens Dataset

This section deals with the evaluation of proposed hybrid recommendation system on MovieLens corpus. Using this dataset, a user-movie bipartite graph is constructed wherein the users are connected to the movies they rated with an edge whose weight is the rating given by a particular user. Given a query user who likes movie q, a recommended movie i is a "surprising movie" if (1) movies i and q share some common genres; and (2) movie i has some new genres that the query movie q does not have. For example, for the query movie "Robin Hood: Men in Tights (Comedy)", "Jack (Comedy and Drama)" is treated as a surprising recommendation; while neither "Spy Hard (Comedy)" or "Raiders of the Lost Ark (Action and Adventure)" is treated as surprising recommendations.

Considering the same user as in section 6.1, who is a comedy fan and whose movie ratings are shown in table 2. Table 4 shows the top 5 movies suggested by the hybrid recommender system. Comparing the results obtained to the recommendations made by content-based collaborative filtering strategy (Table 3), it is not difficult to notice that the top 5 movies recommended using conventional approach consist mainly of comedy movies. On the other hand, niche comedy movies such as "Son in Law (1974)" and "The Flintstones (1994)" do not make it to the top 5 ranking by the hybrid recommender system and popular movies such as "Men in Black (1997)" and "Jack (1996)" emerge instead. That is, the proposed

Table 3: Top 5 movies recommended by content-based CF algorithm

Movies	Genre
The Flintstones (1994)	Children's, Comedy
Son in Law (1974)	Comedy, Horror
The Princess Bride (1987)	Action, Adventure, Comedy, Romance
Star Wars (1977)	Action, Adventure, Romance, Sci-Fi, War
Toy Story (1995)	Animated, Children's, Comedy

Table 4: Top 5 movies recommended by Hybrid-Recommendation System

Movies	Genre
Star Wars (1977)	Action, Adventure, Romance, Sci-Fi, War
The Princess Bride (1987)	Action, Adventure, Comedy, Romance
Toy Story (1995)	Animated, Children's, Comedy
Men in Black (1997)	Action, Adventure, Comedy, Sci-Fi
Jack (1996)	Comedy, Drama

algorithm recommends a wider variety of genres and gives No. 1 rank to "Star Wars (1977)", which scores high on both user preference as well as with the connectivity to other groups and a movie like "The Flintstones (1994)" decreases its rank dramatically. The top movies in table 5 belong to a diverse range of genres even for a user whose preference is comedy movies which means the algorithm makes horizon-broadening recommendations. On average, the "Surprising Gain" of the recommender system on MovieLens dataset is 0.09. An important observation is that as the length of recommendation list increases, the number of average surprising nodes chosen by the hybrid recommender also increases. Thus the proposed algorithm works really well when the number of recommendations required is larger.

The runtime complexity of the hybrid recommender system is $O(n)$ where n is the number of nodes in the user-movie graph. This is because the bridging score computation and the similarity score computation takes $O(n)$ and so does merging the two which is just multiplying both the scores. Thus the hybrid recommender system is very effective and highly scalable due to linear complexity.

7. CONCLUSIONS

The paper aimed at exploring the question if content-based collaborative filtering merged with a heuristic that estimates the utility function to form a hybrid recommender system, improves accuracy in movie recommendations. The baseline method which is the traditional content-based CF algorithm used to predict the ratings of users on MovieLens corpus produced an accuracy of 81% which is far better than a naive estimate. That means 8 out of 10 times a movie is recommended to a user, the user will like it.

The motivation behind creating a hybrid recommender system was to have a "surprise-me" element in movie recommendations. The approach was a hybrid system that merges the conventional method with serendipitous discoveries, that is to find items that are close to the user preferences, while they also have high connectivity to other groups. The system makes recommendations that broaden the horizons of the user, in the sense that it is close enough to his/her current interests to be pleasant, but also towards a new area that the user has not discovered yet. The goal was to automate the process of making surprising yet relevant recommendations.

The advantage of having such a hybrid algorithm other than those described above is that it is (1) parameter-free, (2) effective and (3) fast. It also overcomes the drawbacks of conventional recommender systems that are (1) new-user problem, (2) overspecialization and (3) sparsity.

8. FUTURE WORK

Future work would focus on the implementation of hybrid recommender system on other datasets to be assured of the effectiveness of such a system. Another important future research area would be building such a hybrid system on Hadoop/MapReduce architecture for large scale recommender systems.

9. REFERENCES

[1] http://www.grouplens.org/node/73.

[2] Gediminas Adomavicius and Alexander Tuzhilin. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING*, 17(6):734–749, 2005.

[3] R. Bell, Y. Koren, and C. Volinsky. Modeling relationships at multiple scales to improve accuracy of large recommender systems. In *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 95–104. ACM, 2007.

[4] Ò. Celma and P. Herrera. A new approach to evaluating novel recommendations. In *Proceedings of the 2008 ACM conference on Recommender systems*, pages 179–186. ACM, 2008.

[5] F. Fouss, A. Pirotte, J.M. Renders, and M. Saerens. Random-walk computation of similarities between nodes of a graph with application to collaborative recommendation. *Knowledge and Data Engineering, IEEE Transactions on*, 19(3):355–369, 2007.

[6] Mouzhi Ge, Carla Delgado-Battenfeld, and Dietmar Jannach. Beyond accuracy: evaluating recommender systems by coverage and serendipity. pages 257–260, 2010.

[7] J.L. Herlocker, J.A. Konstan, L.G. Terveen, and J.T. Riedl. Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems (TOIS)*, 22(1):5–53, 2004.

[8] Leo Iaquinta, Marco de Gemmis, Pasquale Lops, Giovanni Semeraro, Michele Filannino, and Piero Molino. Introducing serendipity in a content-based recommender system. In Fatos

Xhafa, Francisco Herrera, Ajith Abraham, Mario Köppen, and José Manuel Benítez, editors, *HIS*, pages 168–173. IEEE Computer Society, 2008.

[9] J. Kamahara, T. Asakawa, S. Shimojo, and H. Miyahara. A community-based recommendation system to reveal unexpected interests. In *Multimedia Modelling Conference, 2005. MMM 2005. Proceedings of the 11th International*, pages 433–438. IEEE, 2005.

[10] Frank McCarey, Mel O. Cinneide, and Nicholas Kushmerick. A recommender agent for software libraries: An evaluation of memory-based and model-based collaborative filtering. pages 154–162, 2006.

[11] Kensuke Onuma, Hanghang Tong, and Christos Faloutsos. TANGENT: a novel, 'surprise me', recommendation algorithm. pages 657–666, 2009.

[12] S. Petrovic, M. Osborne, and V. Lavrenko. Streaming first story detection with application to twitter. In *Proceedings of NAACL*, volume 10. Citeseer, 2010.

[13] U. Shardanand and P. Maes. Social information filtering: algorithms for automating "word of mouth". In *Proceedings of the SIGCHI conference on Human factors in computing systems*, pages 210–217. ACM Press/Addison-Wesley Publishing Co., 1995.

[14] S. Solanki. *Engineering enhancements for movie recommender systems*. PhD thesis, KANSAS STATE UNIVERSITY, 2012.

[15] Cai-Nicolas Ziegler, Sean M. McNee, Joseph A. Konstan, and Georg Lausen. Improving recommendation lists through topic diversification. pages 22–32, 2005.